

Agentic AI Internship

Question Bank & Answer Guide

50 Curated Questions • Topic-wise • Easy / Medium / Hard

Designed for: 2nd-Year B.Tech students targeting Summer Internships

Coverage: LLMs • Prompt Engineering • Tokens • Embeddings • Vector DBs • RAG • LangChain • LangGraph • Agents • Guardrails • MCP • HR

Built from real interview transcripts + current 2026 hiring trends

How to Use This Guide

This guide is built around three principles: short, keyword-dense answers; a difficulty ladder you can climb topic by topic; and a strong overlap with what is actually being asked in interviews right now (drawn from real interview transcripts + current job-market trends for AI/agent roles in 2026).

- **Easy** → definition + 1-line role/use. Should be instant recall.
- **Medium** → comparisons, trade-offs, common variants. Be ready to whiteboard.
- **Hard** → design / debug / failure-mode questions. Tie back to your project.

Tip: when an interviewer asks a theory question, anchor your answer in something you have actually built. "In my WEB-RAG project I used recursive chunking with 200-token overlap because..." beats a textbook definition every time.

Topics Covered

1. LLM Fundamentals (Q1–Q6)
2. Prompt Engineering (Q7–Q11)
3. Tokens & Tokenization (Q12–Q15)
4. Embeddings & Vector Databases (Q16–Q21)
5. RAG — Retrieval-Augmented Generation (Q22–Q28)
6. LangChain (Q29–Q33)
7. LangGraph & AI Agents (Q34–Q39)
8. Guardrails & Safety (Q40–Q43)
9. MCP, Tools & Real-World Agentic Design (Q44–Q46)
10. HR / Behavioural (Q47–Q50)

1. LLM Fundamentals

EASY

Q1. What is an LLM?

Answer: A Large Language Model — a transformer-based neural network trained on massive text corpora that predicts the next token. Examples: GPT-4, Claude, Gemini, Llama.

Keywords: transformer, next-token prediction, pretraining, decoder-only, parameters (B), GPT/Claude/Llama

Q2. What are hallucinations in LLMs?

Answer: When the model generates fluent, confident output that is factually wrong or fabricated. Caused by gaps in training data, statistical pattern-matching, no real-time grounding, or out-of-distribution queries.

Keywords: fabrication, ungrounded, RAG mitigation, lower temperature, citations, fact-checking, guardrails, "I don't know" fallback

MEDIUM

Q3. Difference between a base model and an instruction-tuned (chat) model?

Answer: Base = raw next-token predictor (continues text). Instruction-tuned = base + SFT on instruction-response pairs + RLHF/DPO; it follows commands, refuses unsafe requests, and is what production apps use.

Keywords: pretraining vs SFT, RLHF, DPO, alignment, system prompt, refusal, completion vs chat

Q4. What is temperature, top-p and top-k? When do you tune them?

Answer: Temperature scales token-probability sharpness (0 = deterministic, 1+ = creative). Top-p (nucleus) samples from the smallest set of tokens whose cumulative probability $\geq p$. Top-k samples from the k highest-probability tokens. Use low temp for RAG/extraction, higher for brainstorming.

Keywords: sampling, deterministic, nucleus sampling, creativity vs reliability, RAG $\rightarrow$ temp 0–0.3

HARD

Q5. You have 10 LLMs and want the most accurate answer — how would you design this? (asked in a real round)

Answer: Build an evaluation harness on a golden dataset, score each LLM (accuracy, F1, faithfulness). For runtime: use ensembling — majority vote / self-consistency, an LLM-as-judge to pick the best response, or a router model that sends each query to the best-suited LLM. Mixture-of-Agents stacks weaker models to beat a strong one.

Keywords: ensemble, self-consistency, LLM-as-judge, router, Mixture-of-Agents (MoA), golden eval set, weighted voting

Q6. Explain the transformer architecture in one paragraph.

Answer: Input tokens $\rightarrow$ embeddings + positional encoding $\rightarrow$ stack of transformer blocks. Each block has multi-head self-attention (scaled dot-product: $\text{softmax}(QK^T/\sqrt{d}) \cdot V$) letting every token look at every other token in parallel, followed by a feed-forward network, residual connections and layer norm. Decoder-only models (GPT/Claude) use causal masking so each token only attends to previous tokens.

Keywords: self-attention, Q/K/V, multi-head, positional encoding, residual + layer norm, FFN, causal mask, decoder-only

2. Prompt Engineering

EASY

Q7. What does the structure of a good prompt look like? (real interview question)

Answer: Role / persona → Task / instruction → Context → Examples (few-shot) → Output format → Constraints. Use clear delimiters (###, XML tags, triple backticks) and put the most important instruction last in long prompts (recency bias).

Keywords: role, task, context, few-shot, output format, delimiters, constraints, recency bias

Q8. Zero-shot vs few-shot prompting?

Answer: Zero-shot: just an instruction, no examples. Few-shot: 1–N input/output examples in the prompt to lock in format and style. Few-shot usually wins for structured output and niche formats.

Keywords: in-context learning, exemplars, format anchoring, k-shot

MEDIUM

Q9. What is Chain-of-Thought (CoT) prompting?

Answer: Asking the model to show intermediate reasoning before the final answer ("let's think step by step"). Improves arithmetic, logic, multi-hop QA. Variants: zero-shot CoT, manual CoT (worked examples), self-consistency CoT (sample N chains, majority vote).

Keywords: step-by-step, intermediate reasoning, self-consistency, zero-shot CoT, multi-hop

Q10. What is ReAct prompting?

Answer: Reason + Act. The model alternates Thought → Action (tool call) → Observation → Thought... until it produces a final answer. It is the foundation of most tool-using agents.

Keywords: Thought/Action/Observation loop, tool use, agent loop, scratchpad

HARD

Q11. What is prompt injection and how do you defend against it?

Answer: An attacker hides instructions in user input or retrieved content ("ignore previous instructions and ...") to override the system prompt. Defenses: separate trusted (system) and untrusted (user/tool) channels, input/output classifiers (Llama Guard, Prompt Guard), structured output, allowlists for tool calls, never execute model output as code without validation, and treat retrieved RAG docs as untrusted.

Keywords: prompt injection, indirect injection, instruction hierarchy, Llama Guard, sandboxing, output validation, RAG content = untrusted

3. Tokens & Tokenization

EASY

Q12. What are tokens in the context of LLMs? (asked verbatim)

Answer: The atomic unit an LLM reads/writes — sub-word pieces, not characters or words. Rough English rule: 1 token ≈ 4 characters ≈ 0.75 words. "Tokenization" itself becomes ~3 tokens.

Keywords: subword unit, ~4 chars/token, BPE, vocabulary, input/output unit

Q13. What are token limits / context windows?

Answer: The max tokens (input + output) a model can handle in one call. e.g., GPT-4 Turbo 128k, Claude 200k+. Exceeding it = truncation or error. Larger context = more cost, latency and "lost-in-the-middle" risk.

Keywords: context window, 128k/200k, truncation, lost-in-the-middle, cost ∝ tokens

MEDIUM

Q14. What is tokenization and what algorithms are common?

Answer: The process of converting raw text into integer token IDs the model can consume. Common algorithms: BPE (used by GPT — tiktoken), WordPiece (BERT), SentencePiece / Unigram (Llama, T5).

Keywords: BPE, WordPiece, SentencePiece, tiktoken, vocab size, encode/decode

HARD

Q15. How does tokenization affect cost, latency and multilingual performance in production?

Answer: Pricing is per-1k-tokens (input + output billed separately), so longer prompts = direct cost. Non-English (Hindi, Tamil, code) often tokenizes 2-4× more tokens than English for the same meaning, inflating cost and shrinking effective context. Mitigations: prompt compression, caching (KV / prompt caching), summarization-based memory, smaller-but-cheaper models for routing, and choosing models with multilingual-aware tokenizers.

Keywords: \$/1k tokens, multilingual penalty, prompt caching, KV cache, prompt compression, batching

4. Embeddings & Vector Databases

EASY

Q16. What are embeddings?

Answer: Dense numerical vectors that represent the meaning of text/images/audio. Semantically similar items end up close in vector space. Typical dimension: 384–3072. Examples: OpenAI text-embedding-3, BGE, Cohere Embed, all-MiniLM.

Keywords: dense vector, semantic representation, dimension, embedding model, MTEB

Q17. What is a Vector Database? (asked verbatim)

Answer: A database optimized for storing high-dimensional vectors and running similarity search (cosine / dot / Euclidean) at scale. Examples: Pinecone, Weaviate, Qdrant, Chroma, Milvus, pgvector, FAISS (library, not a server).

Keywords: ANN index, similarity search, Pinecone/Weaviate/Qdrant/Chroma, pgvector, FAISS

MEDIUM

Q18. What is cosine similarity and why is it preferred for text embeddings?

Answer: Measures the cosine of the angle between two vectors: $\cos(\theta) = (A \cdot B) / (||A|| \cdot ||B||)$. Range [-1, 1]; for normalized embeddings, [0, 1]. It ignores magnitude and only compares direction, which matches how semantic meaning is encoded — so a long doc and a short query can still match.

Keywords: cosine sim, dot product, magnitude-invariant, normalized vectors, semantic distance

Q19. What is ANN (Approximate Nearest Neighbour) search?

Answer: Trades a tiny bit of accuracy for huge speed-ups over exact k-NN. Algorithms: HNSW (graph-based, default in most vector DBs), IVF / IVF-PQ, LSH, ScaNN. Tunable via ef_search, nprobe, etc.

Keywords: HNSW, IVF, LSH, ScaNN, recall vs latency, ef_search, nprobe

HARD

Q20. How do you choose an embedding model for a project?

Answer: Match (a) domain — general vs code (CodeBERT) vs biomedical (BioBERT); (b) language coverage; (c) max input tokens; (d) dimension vs cost/latency trade-off; (e) symmetric (semantic similarity) vs asymmetric (query→doc) retrieval; (f) MTEB leaderboard scores; (g) open-source vs API; (h) whether you need on-prem / privacy.

Keywords: MTEB, domain fit, dimension trade-off, asymmetric vs symmetric, max seq len, open vs hosted

Q21. Dense vs sparse embeddings, and what is hybrid search?

Answer: Dense = neural embeddings, capture semantics but can miss exact keywords/IDs. Sparse = BM25 / TF-IDF, great for rare terms and exact matches but no semantics. Hybrid search runs both and fuses the rankings with Reciprocal Rank Fusion (RRF) — usually a 5–15% recall lift, especially on technical/queries with proper nouns.

Keywords: BM25, TF-IDF, dense vs sparse, hybrid search, RRF, recall lift, keyword + semantic

5. RAG — Retrieval-Augmented Generation

EASY

Q22. What is RAG? (asked verbatim, multiple rounds)

Answer: Retrieval-Augmented Generation. At query time you retrieve relevant chunks from an external knowledge base and inject them into the LLM's prompt so it answers grounded in your data. Solves: stale knowledge, private data, and hallucinations — without retraining.

Keywords: retrieval + generation, grounding, no retraining, private data, fights hallucination

Q23. What are the components of a basic RAG pipeline?

Answer: Ingestion → chunking → embedding → vector store. At query time: embed query → retrieve top-k → (optional rerank) → assemble prompt with context → LLM generates answer with citations.

Keywords: ingest, chunk, embed, store, retrieve, rerank, augment prompt, generate, cite

MEDIUM

Q24. What is chunking and why does it matter?

Answer: Splitting documents into smaller pieces before embedding. Strategies: fixed-size, recursive character, sentence/paragraph, semantic, markdown-header-aware. Trade-off: too small loses context, too big dilutes the embedding and wastes context window. Add 10–20% overlap to preserve cross-chunk meaning.

Keywords: fixed/recursive/semantic chunking, overlap, chunk size 200–800 tokens, granularity vs context

Q25. What is a reranker and where does it sit in the pipeline?

Answer: A second-stage model that re-scores the top-k retrieved chunks against the query. Usually a cross-encoder (Cohere Rerank, BGE-reranker, Jina) — slower but much more accurate than the bi-encoder used for initial retrieval. Pattern: retrieve top-50, rerank to top-5.

Keywords: cross-encoder, two-stage retrieval, top-k → top-n, Cohere Rerank, BGE-reranker, precision boost

Q26. How would you debug a RAG system that gives bad answers?

Answer: Isolate the layer: is retrieval missing the right chunk (tweak chunking, embeddings, hybrid search, metadata filters), or is the LLM failing to use it (prompt, citations, lost-in-the-middle, smaller k)? Add evals (RAGAS), log retrieved chunks, compare with golden Q-A pairs, then fix the lowest-scoring stage first.

Keywords: decompose retrieval vs generation, RAGAS, faithfulness, query rewriting, reranking, metadata filter

HARD

Q27. What is HyDE (Hypothetical Document Embeddings)?

Answer: Instead of embedding the user's short query, ask the LLM to generate a hypothetical answer/document, embed THAT, and retrieve nearest chunks. Works because answers and

answer-shaped passages live closer together in vector space than questions and answers do. Helps zero-shot retrieval and short queries.

Keywords: HyDE, query expansion, hypothetical doc, query↔doc style mismatch, zero-shot retrieval

Q28. How do you evaluate a RAG system?

Answer: RAGAS metrics: faithfulness (is the answer grounded in retrieved context?), answer relevancy, context precision, context recall. Plus retrieval IR metrics — MRR, NDCG@k, hit-rate. Use an LLM-as-judge on a curated golden set; track regression over time. End-to-end and per-stage evals.

Keywords: RAGAS, faithfulness, context precision/recall, MRR, NDCG, LLM-as-judge, golden dataset

6. LangChain

EASY

Q29. What is LangChain?

Answer: An open-source framework for building LLM-powered apps. Provides high-level abstractions — prompts, chains, retrievers, memory, tools, agents — plus integrations with most LLM providers and vector DBs.

Keywords: framework, abstractions, integrations, chains/agents/retrievers/memory, LCEL

Q30. What is a Chain in LangChain?

Answer: A composed sequence of calls — to an LLM, a tool, a retriever, a parser, etc. Modern LangChain uses LCEL (LangChain Expression Language) and the `|` (pipe) operator: `prompt | llm | parser`.

Keywords: LCEL, pipe operator, composition, Runnable, prompt → llm → parser

MEDIUM

Q31. What types of memory does LangChain provide?

Answer: ConversationBufferMemory (full history), BufferWindowMemory (last k turns), SummaryMemory (LLM-summarised history), EntityMemory (per-entity facts), VectorStoreRetrieverMemory (semantic recall). Choice depends on token budget vs fidelity.

Keywords: buffer, window, summary, entity, vector memory, token budget vs fidelity

Q32. What are Document Loaders and Text Splitters in LangChain?

Answer: Loaders ingest data from many sources — PDF, web, Notion, CSV, S3, Confluence — into a common Document object. Splitters chunk that text: RecursiveCharacterTextSplitter (default), TokenTextSplitter, MarkdownHeaderTextSplitter, SemanticChunker.

Keywords: DocumentLoader, RecursiveCharacterTextSplitter, TokenTextSplitter, MarkdownHeaderTextSplitter, ingestion

HARD

Q33. What are the limitations of LangChain and when would you NOT use it?

Answer: Heavy abstractions hide what's actually being sent to the LLM, fast-moving API breaks across versions, performance overhead, and debugging complex chains is painful. For simple production apps, calling the provider SDK directly is cleaner. For complex stateful agents, LangGraph (lower-level, explicit state) is preferred. LlamaIndex is often better for pure-RAG.

Keywords: leaky abstraction, breaking changes, debugging, alternatives: native SDK / LangGraph / LlamaIndex / DSPy

7. LangGraph & AI Agents

EASY

Q34. What is LangGraph?

Answer: A library (by the LangChain team) for building stateful, multi-actor LLM apps as a graph. Nodes = functions/agents, edges = control flow. Unlike DAG chains, LangGraph supports cycles (loops, retries, agent-loop), persistence, and human-in-the-loop.

Keywords: stateful graph, nodes + edges, cycles, checkpoint, human-in-the-loop, multi-agent

Q35. What is an AI agent?

Answer: An LLM-driven system that autonomously plans, decides, and uses tools to achieve a goal — usually in a loop: Observe → Think → Act → Observe... Has a planner, memory, a set of tools, and a stopping condition.

Keywords: autonomy, planning, tool use, ReAct loop, memory, goal-directed

MEDIUM

Q36. What's the difference between a chain and an agent?

Answer: A chain has a fixed, predetermined sequence of steps — same path every run. An agent uses the LLM at runtime to decide what to do next (which tool, with what arguments, when to stop). Chains are predictable and cheaper; agents are flexible but harder to control and evaluate.

Keywords: static vs dynamic flow, deterministic vs LLM-routed, tool selection, control vs flexibility

Q37. What are tools in agentic frameworks?

Answer: External functions an agent can call — web search, DB query, calculator, code execution, custom APIs. Each is registered with a name, description, and input schema (Pydantic / JSON schema). The LLM picks one via native function/tool calling and supplies arguments as JSON.

Keywords: function calling, tool schema, Pydantic, JSON schema, name + description matter, OpenAI/Anthropic tool use

HARD

Q38. Sketch the architecture of a multi-agent system.

Answer: Multiple specialised agents (e.g., Planner, Researcher, Coder, Critic) coordinated by either a Supervisor agent (hierarchical) or peer-to-peer message passing (network). Shared state object (LangGraph State / blackboard), explicit handoff protocol, role-specific system prompts, and a stopping condition. Frameworks: LangGraph, CrewAI, AutoGen, OpenAI Swarm.

Keywords: supervisor, hierarchical vs network, shared state, handoff, role prompts, CrewAI/AutoGen/LangGraph/Swarm

Q39. How does state management work in LangGraph?

Answer: You define a State schema (TypedDict / Pydantic). Each node receives the current state and returns a partial update. Reducers (e.g., `add\_messages`) define how updates merge into the state. A Checkpointer (e.g., SQLite, Postgres) persists state per thread_id, enabling pause/resume, time-travel debugging, and human-in-the-loop interrupts.

Keywords: TypedDict state, reducer, add_messages, checkpoint, thread_id, time travel, interrupt

8. Guardrails & Safety

EASY

Q40. What are guardrails in LLM applications?

Answer: Programmatic safety/policy controls applied to inputs and outputs of an LLM: PII detection, toxicity / hate filters, prompt-injection detection, jailbreak detection, topic restrictions, schema validation, profanity. Tools: NeMo Guardrails (NVIDIA), Guardrails AI, Llama Guard, Prompt Guard.

Keywords: input/output filter, PII, toxicity, jailbreak, schema validation, NeMo Guardrails, Llama Guard

MEDIUM

Q41. How do you handle hallucinations in production?

Answer: Ground every answer with RAG + citations, force structured output (JSON schema), allow an explicit "I don't know" path, lower temperature for factual tasks, verify the answer against retrieved context (faithfulness check / NLI model), and route low-confidence outputs to human review.

Keywords: RAG grounding, citations, structured output, low temp, faithfulness check, human-in-the-loop, confidence threshold

Q42. What is content moderation in an LLM system and where do you apply it?

Answer: Filtering harmful content (hate, violence, sexual, self-harm, illegal) on BOTH input (block bad user prompts) and output (block bad model generations). Use moderation APIs (OpenAI Moderation, Perspective), classifiers (Llama Guard), policy-aware system prompts, and refusal-trained models.

Keywords: two-sided moderation, OpenAI Moderation API, Perspective, Llama Guard, policy prompts

HARD

Q43. Design a guardrail layer for a customer-support agent that has access to user data.

Answer: Input pipeline: PII redaction → prompt-injection classifier → topic / scope check (in-domain?). Generation: tool-call allowlist, schema-validated outputs, RAG-only answers (no free generation on policy questions). Output pipeline: faithfulness check vs retrieved context, toxicity filter, PII leak check, structured-format validator. Failover: low-confidence or refused → escalate to human; full request/response logging + observability for audits.

Keywords: two-pipeline design, PII in/out, scope check, tool allowlist, faithfulness, escalation, observability, audit log

9. MCP, Tools & Real-World Agentic Design

EASY

Q44. What is MCP (Model Context Protocol)?

Answer: An open protocol introduced by Anthropic that standardises how LLMs connect to external tools, data sources and prompts — often described as "USB-C for AI". An MCP server exposes resources, tools and prompts; any MCP-compatible client (Claude Desktop, Cursor, custom agents) can plug in without bespoke integration code.

Keywords: Model Context Protocol, Anthropic, MCP server/client, tools/resources/prompts, standard interface, plug-and-play

MEDIUM

Q45. What is function / tool calling and how does it work?

Answer: You give the LLM a list of functions with names, descriptions and JSON schemas. The model decides whether to call one, and emits structured JSON args matching the schema. Your code executes the function and feeds the result back. Native in OpenAI, Anthropic, Gemini, Mistral. It's the mechanism behind virtually every agent.

Keywords: structured JSON output, schema, native API support, basis of agents, parallel tool calls

HARD

Q46. Design an AI agent that auto-responds to incoming customer emails. (real interview scenario)

Answer: Trigger: new email webhook. Step 1 — classify intent (sales / support / complaint / spam) and sentiment via LLM. Step 2 — entity extraction (order ID, product, account). Step 3 — RAG over

product docs, FAQ, past tickets. Step 4 — generate draft response with citations. Step 5 — guardrails: PII check, tone check, policy check. Step 6 — confidence-based routing: high → auto-send + log; medium → human approval queue; low/escalation → assign to agent. Step 7 — write back to CRM, update memory. Stack: n8n / LangGraph orchestration, vector DB for KB, MCP for CRM/email, observability (Langfuse / LangSmith).

Keywords: intent classification, entity extraction, RAG, guardrails, confidence routing, human-in-loop, n8n/LangGraph, CRM via MCP, Langfuse

10. HR / Behavioural

EASY

Q47. What are you currently learning? (asked in nearly every round)

Answer: Be specific and recent. Sample answer: "Right now I'm building a multi-agent research assistant in LangGraph with checkpointed state and human-in-the-loop. Last week I added hybrid retrieval (BM25 + dense) and started evaluating it with RAGAS. Next I want to wire it through MCP so I can swap clients." Mention a GitHub link if asked.

Keywords: specificity, recent activity, project name, framework, what's next, GitHub portfolio

Q48. Tell me about a project you've built. (universal)

Answer: Use STAR-ish structure: Problem → Your role → Stack (LLM, framework, vector DB, deployment) → 2 hard challenges and how you solved them (e.g., chunking, hallucinations, latency) → Outcome / numbers / what you learned. Always tie back to the role you're applying for.

Keywords: STAR, problem-solution-impact, stack, challenges, metrics, lessons learned, role-tied

Q49. Where do you see yourself in 3–5 years? (asked verbatim)

Answer: Honest + growth-oriented + specific. Example: "Working as an AI engineer focused on agentic systems — building production-grade multi-agent products, contributing to open source, and growing into a tech lead role. Short-term I want to deepen my systems-level skills (eval, observability, deployment) so I'm not just prompt-stitching."

Keywords: growth path, specific role, contribution, learning goals, avoid generic 'manager' answers

Q50. Why agentic AI and why this company?

Answer: Two-part: (1) Why the field — agents are the next interface, you've felt the leap from chatbots to systems that can plan and act, you've built X. (2) Why this company — concrete: a product they ship, a blog post they wrote, a tech-stack overlap, a problem you'd want to own. Avoid "because it's exciting / future" with no specifics.

Keywords: genuine motivation, concrete examples, company-specific research, your built work, value-fit

Final Tips Before Your Interview

- ▶ **Always anchor theory in your project.** "In WEB-RAG I used recursive chunking with 200-token overlap because..." beats a textbook definition.
- ▶ **Know your stack cold.** Be ready to explain WHY you picked the LLM, vector DB, embedding model, framework. "Default" is not an answer.
- ▶ **Honesty about gaps wins.** "I haven't used X but here's what I'd do based on Y" beats faking it. Multiple transcripts show interviewers explicitly rewarding this.
- ▶ **Brush up DSA basics** — hashmaps, arrays, two-pointer. Even agentic-AI rounds often slip in a coding question.
- ▶ **Polish backend + system design.** FastAPI, REST, SQL vs NoSQL, indexing, CORS, ACID — these came up in real rounds for AI roles.
- ▶ **Have your project's GitHub + live link ready.** And know your own code — "explain this function" is a common gotcha.

Good luck! Build, ship, iterate.